CSE - 214

Clean Coding Practices

Prepared by
Ishrat Jahan

Any fool can write code that a computer can understand.

Good programmers write code that humans can understand.

What is/Why Clean Code?

1. Readability
2. Maintainability
3. Collaboration
4. Bug Reduction
5. Efficiency

Key Factors

- Effectiveness
- Efficiency
- Simplicity

Best Practices and Principles

1. **Format and Syntax**

- Keep it consistent
- Follow the standards
- Take care of **indentations, spaces, and cases**

```
// bad indentation and spacing
const myFunc=(number1,number2)=>{
const result=number1+number2;
return result;
}

// good indentation and spacing
const myFunc = (number1, number2) => {
    const result = number1 + number2
    return result
}
```

Best Practices and Principles

2. Naming Conventions

- Use **meaningful, self-explanatory** names

```
int fxn(int a[], int n) {
    int i, x = 0;
    for (i = 0; i < n; i++) {
        if (a[i] > x) {
            x = a[i];
        }
    }
    return x;
}
```

```
int findLargestElement(int array[], int size) {
    int i, largest = array[0];
    for (i = 1; i < size; i++) {
        if (array[i] > largest) {
            largest = array[i];
        }
    }
    return largest;
}
```

Best Practices and Principles

3. Comments

Comments are usually created when the author realizes that his or her code isn't intuitive or obvious.

In such cases, comments are like a deodorant masking the smell of fishy code.

The best comment is a good name for a method or class

Comments can be useful:

- To explain **why**
- When explaining complex algorithms

Comment as deodorant — bad

```
void proc(Order o) {
// check if order is valid
if (o.items == null || o.amt < 0)
throw new Exception("bad");
// apply 10% discount if eligible
if (o.loyalty) o.amt *= 0.9;
}
```

Comments exist because names
proc, o, amt hide the intent

Good names — no comment needed

```
void processOrder(Order order) {
validateOrder(order);
applyLoyaltyDiscount(order);
}

void validateOrder(Order order) {
if (order.items == null || ...
}
```

The method name IS the comment
No deodorant needed

When comments ARE welcome

Explain the why — good

```
// Stripe rejects amounts > 999999.
// Cap prevents silent API failure.
if (amount > 999999)
amount = 999999;

}
```

The name says what; the comment explains a non-obvious constraint

Complex algorithm — good

```
// Luhn algorithm: validates card
// numbers by doubling every 2nd
// digit from the right.
boolean isValidCard(String n) {
int sum = 0; boolean alt = false;
...
}
```

Algorithm isn't obvious; comment links to the known standard

Best Practices and Principles

4. **SRP (Single Responsibility Principle)**

- A function should do one thing and do it well
- Keep functions and methods concise

A function doing a lot of things:

```java
public void printGrade(String name, int score) {
    if (score < 0 || score > 100)
        throw new IllegalArgumentException("Score must be 0–100");
    String grade = score >= 90 ? "A" : score >= 75 ? "B" : score >= 60 ? "C" : "F";
    String result = String.format("Student: %-15s Score: %d  Grade: %s", name, score, grade);
    System.out.println(result);
}

// Caller just calls print
public void handleGrade(String name, int score) {
    printGrade(name, score);
}
```

All functions following SRP:

```java
public void validate(int score) {
    if (score < 0 || score > 100)
        throw new IllegalArgumentException("Score must be 0–100");
}

public String toLetterGrade(int score) {
    return score >= 90 ? "A" : score >= 75 ? "B" : score >= 60 ? "C" : "F";
}

public String formatGrade(String name, int score, String grade) {
    return String.format("Student: %-15s Score: %d  Grade: %s", name, score, grade);
}

public void printGrade(String name, int score, String grade) {
    String result = formatGrade(name, score, grade);
    System.out.println(result);
}

// Caller orchestrates each responsibility
public void handleGrade(String name, int score) {
    validate(score);
    String grade  = toLetterGrade(score);
    printGrade(result);
}
```

Best Practices and Principles

5. **DRY (Do not Repeat Yourself)**

- A function should do one thing and do it well
- Keep functions and methods concise

```
// BAD — email check copy-pasted wherever it's needed

class RegistrationService {
    void register(String email) {
        if (!email.contains("@") || email.length() < 5)  // copy 1
            throw new IllegalArgumentException("Invalid email");
        // ...
    }
}

class ProfileUpdateService {
    void updateEmail(String email) {
        if (!email.contains("@") || email.length() < 5)  // copy 2
            throw new IllegalArgumentException("Invalid email");
        // ...
    }
}

class AdminUserService {
    void createAdmin(String email) {
        if (!email.contains("@") || email.length() < 5)  // copy 3
            throw new IllegalArgumentException("Invalid email");
        // ...
    }
```

Refactored Version:

```java
// GOOD — validation rule lives in exactly one class

class EmailValidator {
    public boolean isValid(String email) {
        if (email == null)            return false;
        if (!email.contains("@"))     return false;
        if (email.length() < 5)       return false;
        // New rule added here once — applies everywhere automatically
        if (!email.endsWith(".com") && !email.endsWith(".org"))
            return false;
        return true;
    }
}

class RegistrationService {
    private final EmailValidator emailValidator;

    void register(String email) {
        if (!emailValidator.isValid(email))
            throw new IllegalArgumentException("Invalid email");
        // ...
```

Best Practices and Principles

6. Conciseness vs Clarity

```
// Example 1: Concise function
const countVowels = s => (s.match(/[aeiou]/gi) || []).length;
console.log(countVowels("hello world"));
```

```
// Example 2: More verbose and clearer function
function countVowels(s) {
  const vowelRegex = /[aeiou]/gi;
  const matches = s.match(vowelRegex) || [];
  return matches.length;
}

console.log(countVowels("hello world"));
```

Best Practices and Principles

7. **Reusability**

   - Check the previous example

8. **Single Source of Truth**

   - Avoid hard-coding numbers
   - Ensure that constants or or config is stored only one place in the codebase

```
// BAD — 0.20 is scattered everywhere, meaning unknown
class OrderService {
    double calculateTax(double price) {
        return price * 0.20;   // magic number
    }
}

class InvoiceService {
    double calculateTax(double price) {
        return price * 0.20;   // same magic number, different file
    }
}

class ReceiptService {
    double calculateTax(double price) {
        return price * 0.20;   // and again
    }
}
```

```
public class TaxConfig {

    // Declared at class level — accessible anywhere as TaxConfig.VAT_RATE
    public static final double VAT_RATE = 0.20;
```

Best Practices and Principles

9. Documentation

- Code tells you *how* — docs tell you *why*
- You forget your own code in 3 months
- New team members onboard faster
- "The best time to document is when you write the code"
- Things to document: Feature details, why the particular business logic was adopted, why this particular API or third party library was used, constraints, testing status, corner cases, scope of improvement, etc.

10. Refactoring



Happy Coding!